Array of Objects

Simple Syntax Notes — JavaScript · TypeScript · Golang

JavaScript

1 Create a single object

Use curly braces {}. Each property is a key: value pair.

```
let car = {
  brand: "Toyota",
  year:  2023
};
```

No type needed — JavaScript figures out the shape at runtime.

2 Create an array of objects

Wrap multiple objects inside square brackets []. Each object is separated by a comma.

```
let cars = [
  { brand: "Toyota", year: 2023 },
  { brand: "Honda",  year: 2022 },
  { brand: "Tesla",  year: 2024 }
];
```

Think of it as: array = a box, objects = items inside the box.

3 Access a value

Use the index to pick an object, then dot-notation to get a property.

```
cars[0].brand  // "Toyota"  <- first item
cars[1].year   // 2022      <- second item
cars[2].brand  // "Tesla"   <- third item
```

Indexes start at 0, not 1.

4 Loop through all objects

Use forEach to visit every object one by one.

```
cars.forEach(car => {
  console.log(car.brand, car.year);
});
// Toyota 2023
// Honda  2022
// Tesla  2024
```

car is just a name you choose — it represents each object as the loop runs.

5 Add a new object

Use push() to add an object to the end of the array.

```
cars.push({ brand: "BMW", year: 2024 });

console.log(cars.length); // 4
```

push() always adds to the end. Use unshift() to add to the front.

TypeScript

1 Define a type first

Before creating objects, define their shape using type. This is the blueprint.

```
type Car = {
  brand: string;
  year:  number;
};
```

This is unique to TypeScript — JavaScript has no type keyword.

2 Create a single object with type

Attach : Car after the variable name. TypeScript will check the shape matches.

```
let car: Car = {
  brand: "Toyota",
  year:  2023
};

// Wrong type? TypeScript gives an error:
// year: "2023"  <- Error! string is not assignable to number
```

TypeScript catches mistakes before you even run the code.

3 Create an array of objects with type

Write Car[] to say "array of Car objects". The [] means array.

```
let cars: Car[] = [
  { brand: "Toyota", year: 2023 },
  { brand: "Honda",  year: 2022 },
  { brand: "Tesla",  year: 2024 }
];
```

Car[] = "an array where every item must be a Car".

4 Access a value

Same as JavaScript — index + dot-notation. But now your editor shows autocomplete!

```
cars[0].brand  // "Toyota" – editor knows this is a string
cars[1].year   // 2022    – editor knows this is a number

// Typo? TypeScript catches it:
// cars[0].bran  <- Error! Property 'bran' does not exist on Car
```

This is the big win — errors at write time, not at runtime.

5 Loop through all objects

Identical to JavaScript, but car is typed as Car automatically.

```
cars.forEach((car: Car) => {
  console.log(car.brand, car.year);
});
// Toyota 2023
// Honda  2022
// Tesla  2024
```

You can omit : Car here — TypeScript infers it from the array type.

Golang

1 Define a struct

In Go, the blueprint is called a struct. Define it outside of any function.

```
type Car struct {
    Brand string
    Year  int
}
```

No semicolons. Each field is written as: Name then Type.

2 Create a single struct instance

Use := to declare and assign at once. Fill fields with FieldName: value.

```
car := Car{
    Brand: "Toyota",
    Year:  2023,
}

fmt.Println(car) // {Toyota 2023}
```

:= is Go's shorthand for declare + assign. No let or var needed here.

3 Create a slice of structs

A slice is Go's version of an array. Write []Car to mean slice of Car.

```
cars := []Car{
    {Brand: "Toyota", Year: 2023},
    {Brand: "Honda",  Year: 2022},
    {Brand: "Tesla",  Year: 2024},
}
```

Inside the slice, you can skip the struct name — Go knows the type from []Car.

4 Access a value

Same index + dot-notation pattern as JS/TS.

```
cars[0].Brand  // "Toyota"
cars[1].Year   // 2022
cars[2].Brand  // "Tesla"

fmt.Println(cars[0].Brand) // Toyota
```

Go field names start with a capital letter (exported). Lowercase = private to the package.

5 Loop through all structs

Use for range. The _ discards the index when you don't need it.

```
for _, car := range cars {
    fmt.Println(car.Brand, car.Year)
}
// Toyota 2023  // Honda 2022  // Tesla 2024

// Need the index too? Use i:
for i, car := range cars {
    fmt.Println(i, car.Brand)
}
```

Go has only one loop keyword — for. No forEach, no while. Just for.

Quick comparison

Step	JavaScript	TypeScript	Golang
Blueprint	`none (dynamic)`	`type Car = {}`	`type Car struct {}`

Step	JavaScript	TypeScript	Golang
Single object	`let car = {}`	`let car: Car = {}`	`car := Car{}`
Array/slice	`let cars = []`	`let cars: Car[] = []`	`cars := []Car{}`
Access	`cars[0].brand`	`cars[0].brand`	`cars[0].Brand`
Loop	`.forEach(car =>)`	`.forEach((car:Car)=>)`	`for _, car := range`